

Programming with floating point in RISC-V

(extension `f` for single precision, `d` for double, `q` for quad)

(reference: text Section 3.5, [RISC-V instruction set manual](#))

separate set of 32 registers

- named **f0** through **f31** in assembly language
- each register 32, 64, or 128 bits

arithmetic instructions

- “**fadd.s**”, “**fadd.d**”, “**fmul.s**”, “**fmul.d**”, “**fsub.s**”, “**fsub.d**”, “**fdiv.s**”, “**fdiv.d**”, and others, including **fused** operations ...
- e.g. “**fadd.s f0, f1, f2**” adds the **single precision** floating point values in registers **f1** and **f2** and stores the result in register **f0**

examples of other useful instructions

“**fmv.w.x**” (copies to a floating point reg. from an integer reg.)

“**fcvt.s.w**” (does the same copy **but also converts format**)

“**flw**” (loads a single precision value into a floating point reg.)

“**fmv.x.w**”, “**fcvt.d.w**”, “**fcvt.s.d**”, “**fcvt.d.s**”, “**fsw**”, “**fld**”, ...

see [sample floating point program on Canvas site](#)

use of single vs. double vs. quad precision?

- single precision: possible speed advantage, and values occupy less memory space
- double precision: more significant digits and greater range
- quad precision: mainly used currently only in mathematical library routines, for greater range and precision of intermediate results

sometimes, even single precision is “too much” ...

- IEEE 754 binary16 format (RISC-V extension `Zfh')
(sign bit, 5 bit exponent field, 10 bit fraction field)
- an alternative 16-bit format is the bfloat16 ("brain floating point") format, developed by Google Brain (now merged into Google DeepMind) (RISC-V `BF16' extensions)
(sign bit, 8 bit exponent field, 7 bit fraction field)
- there are other formats with even fewer bits
(8 bits or even just 4 bits!)

Operations on floating point numbers

overflow/underflow and roundoff error

overflow/underflow occurs when number of bits in exponent field too small to represent required exponent

roundoff error occurs when can't represent result exactly with the limited number of bits available in the fraction field

addition/subtraction of floating point numbers

basic algorithm:

- scale number with *smallest* exponent so that exponents match
- add/subtract “**significands**” (i.e., the digits, including the leading digit)
- normalize (checking for overflow/underflow)
- round
- re-normalize if necessary

keep extra digits during operation called the **guard** and **round**, as well as an extra bit called the **sticky bit** that keeps track of whether or not there were non-zero digits to the right of the round

example with 3-digit decimal numbers

$$\begin{array}{r} 1.00 \times 10^2 \text{ (100)} \\ - 3.43 \times 10^{-1} \text{ (0.343)} \end{array}$$

scale so that exponents match:

$$\begin{array}{r} 1.0000 \times 10^2 \\ - \underline{0.0034 \times 10^2} \quad \text{(with guard and round)} \\ 0.9966 \times 10^2 \end{array}$$

normalize, and round, to get 9.97×10^1

floating point operations are **not associative**

- consider previous example and the expression $-99.7 + 100 - 0.343$
- when writing code that does a lot of floating point calculations, *sometimes have to carefully structure calculations to minimize the accumulated roundoff error*

multiplication

basic algorithm:

- add exponents by adding contents f_1 and f_2 of the two exponent fields, and then subtracting bias; works since:

$$f_1 + f_2 - \text{bias} = (f_1 - \text{bias}) + (f_2 - \text{bias}) + \text{bias}$$

- multiply significands
- normalize (checking for underflow/overflow)
- round
- re-normalize if necessary
- set appropriate sign of product

division

high-performance (yet correct) implementations are non-trivial (see text Section 3.9 for an interesting story!)

parallelism

many important applications operate on vectors or arrays of floating point (or integer) values, can speed up such applications through use of “**vector**” (or “**SIMD**”) instructions (more on this later ...)